NAME:- Farooqui Shahzeb Ahmed

ROLL.NO:- SYA-29

ASSINGMENT:-07

AIM:-**Implement and analyze various probability distributions in Python, including Binomial, Poisson, and Normal distributions.**

THEORY:-

Probability Distributions

A probability distribution is a mathematical function that describes the likelihood of obtaining the possible values that a random variable can take. It provides a complete picture of all possible outcomes and their associated probabilities. Probability distributions are broadly categorized into two types: discrete distributions, which apply to outcomes that take on countable values such as the number of heads in a coin flip, and continuous distributions, which apply to outcomes that can take on any value within a given range, such as the height of a person. Understanding probability distributions is fundamental to statistics, data science, and machine learning, as they underlie nearly every method of inference, prediction, and decision-making used in practice.

Binomial Distribution

The Binomial distribution is one of the most fundamental discrete probability distributions. It models the number of successes in a fixed number of independent trials, where each trial has exactly two possible outcomes — success and failure — and the probability of success (p) remains constant across all trials. The distribution is parameterized by n (the number of trials) and p (the probability of success in each trial). The probability mass function is given by P(X = k) = C(n,k) * p^k * (1-p)^(n-k), where C(n,k) is the binomial coefficient "n choose k". The mean of the Binomial distribution is n*p and the variance is n*p*(1-p). Common real-world examples include the number of defective items in a production batch, the number of patients who respond to a treatment, or the number of heads when flipping a coin multiple times.

Poisson Distribution

The Poisson distribution is a discrete probability distribution that models the number of events occurring within a fixed interval of time or space, given that these events occur with a known constant mean rate and independently of each other. It is parameterized by a single value λ (lambda), which represents the average number of events per interval. The probability mass function is P(X = k) = (e^(-λ) * λ^k) / k!. A unique and important property of the Poisson distribution is that its mean and variance are both equal to λ. The Poisson distribution naturally arises when events are rare, random, and independent, making it incredibly useful in fields like telecommunications, healthcare, manufacturing quality control, and traffic engineering.

Normal Distribution

The Normal distribution, also known as the Gaussian distribution, is arguably the most important probability distribution in all of statistics and data science. It is a continuous distribution characterized by its iconic symmetric, bell-shaped curve and is fully described by two parameters: the mean (μ), which determines the center of the distribution, and the standard deviation (σ), which controls its spread. The probability density function is $f(x) = (1/(\sigma\sqrt{2\pi})) * e^{(-(x-\mu)^2/(2\sigma^2))}$. The Normal distribution follows the empirical rule (68-95-99.7 rule): approximately 68% of data falls within one standard deviation of the mean, 95% within two, and 99.7% within three. Its mathematical tractability, connection to the Central Limit Theorem, and prevalence in natural phenomena make it indispensable in statistics, machine learning, and scientific research.

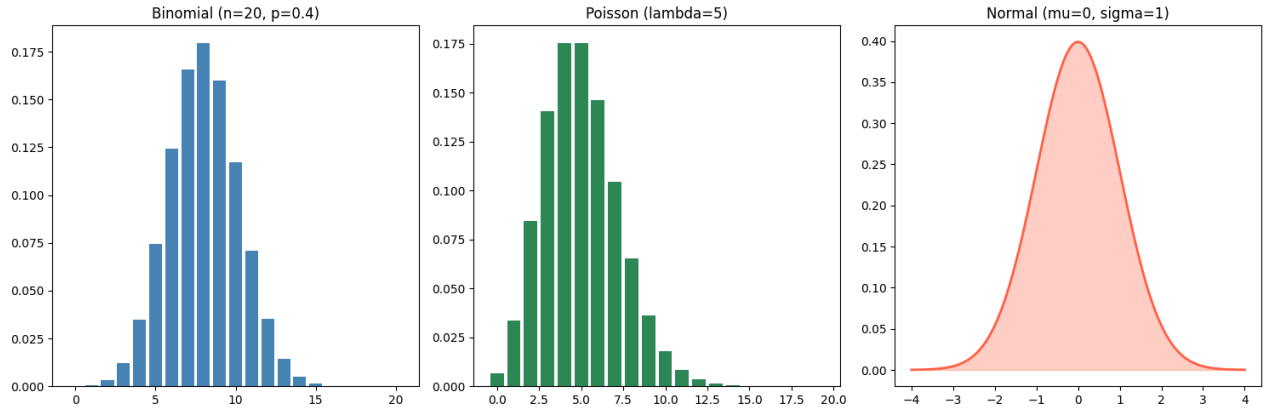
Q1 **Difference Between Binomial, Poisson, and Normal Distributions**

ANS:-

Feature	Binomial Distribution	Poisson Distribution
Type	Discrete	Discrete
Parameters	n (number of trials), p (probability of success)	λ (lambda - average rate of events)
Conditions	Fixed number of trials, independent trials, two outcomes (success/failure), constant p	Events occur at a constant mean rate,
PMF/PDF	PMF: P(X=k) = C(n,k) * p^k * (1-p)^(n-k)	PMF: P(X=k) = (e^(-λ) * λ^k) / k!
Mean	n * p	λ
Variance	n * p * (1-p)	λ
Shape	Can be skewed (p ≠ 0.5) or symmetric (p = 0.5)	Skewed for small λ, approaches symm
Typical Use	Number of successes in a fixed number of trials (e.g., coin flips, defective items)	Number of events in a fixed interval (e.
Approximation	Approximated by Normal for large n, by Poisson for large n and small p	Approximated by Normal for large λ

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import binom, poisson, norm

n, p = 20, 0.4
x_bin = np.arange(0, n+1)
y_bin = binom.pmf(x_bin, n, p)
print(f"Binomial  -> Mean: {n*p}, Variance: {n*p*(1-p)}")

lam = 5
x_poi = np.arange(0, 20)
y_poi = poisson.pmf(x_poi, lam)
print(f"Poisson   -> Mean: {lam}, Variance: {lam}")

mu, sigma = 0, 1
x_norm = np.linspace(-4, 4, 300)
y_norm = norm.pdf(x_norm, mu, sigma)
print(f"Normal    -> Mean: {mu}, Variance: {sigma**2}")

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].bar(x_bin, y_bin, color="steelblue")
axes[0].set_title(f"Binomial (n={n}, p={p})")
axes[1].bar(x_poi, y_poi, color="seagreen")
axes[1].set_title(f"Poisson (lambda={lam})")
axes[2].plot(x_norm, y_norm, color="tomato", linewidth=2)
axes[2].fill_between(x_norm, y_norm, alpha=0.3, color="tomato")
axes[2].set_title(f"Normal (mu={mu}, sigma={sigma})")
plt.tight_layout()
plt.show()
```

```
Binomial  -> Mean: 8.0, Variance: 4.8
Poisson   -> Mean: 5, Variance: 5
Normal    -> Mean: 0, Variance: 1
```



Q2 **What is the real-world significance of the Poisson distribution?**

ANS:- The Poisson distribution holds immense practical significance across a wide range of real-world domains because it naturally models the occurrence of rare, discrete, and independent events over a fixed period of time or unit of space. Its simplicity — requiring only a single parameter λ — combined with its remarkable applicability to spontaneous events makes it one of the most practically valuable distributions in applied statistics and engineering.

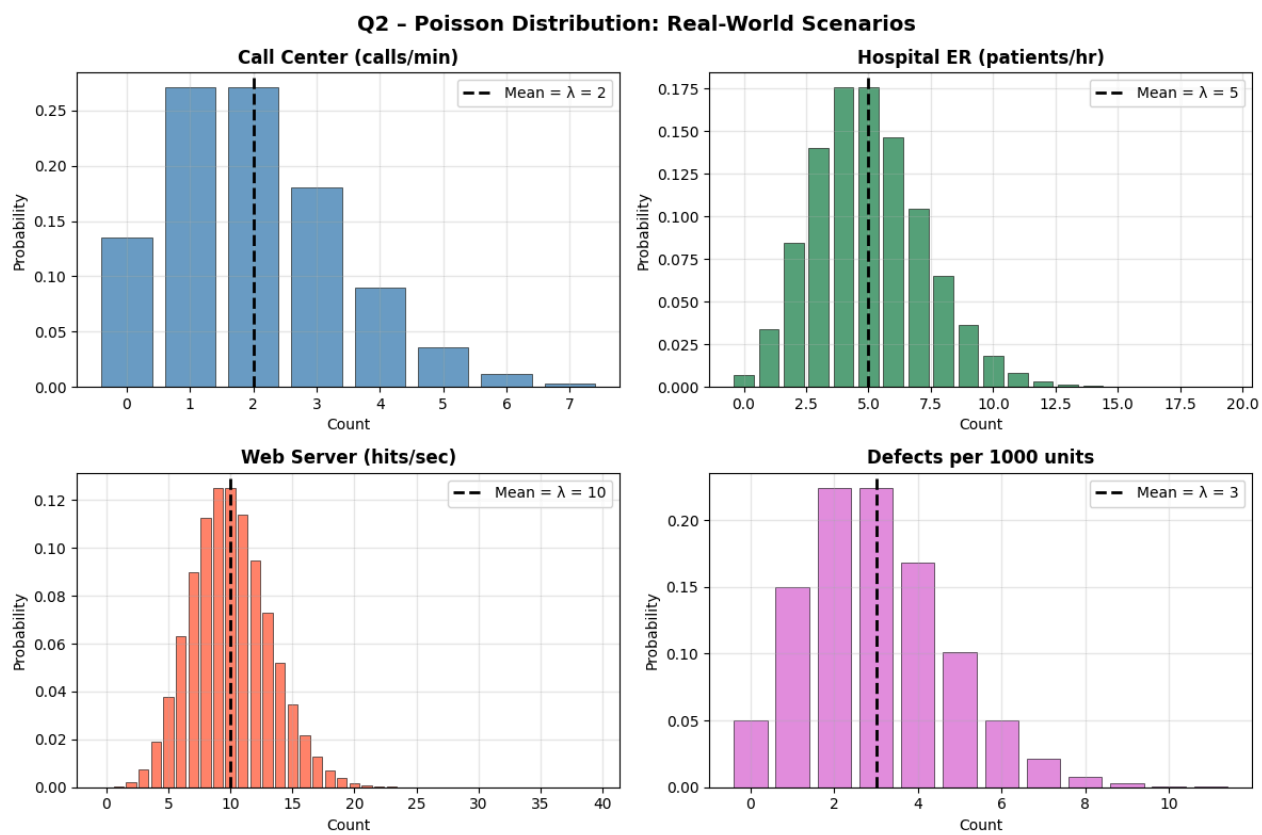
In telecommunications and call center management, the Poisson distribution is used to model the number of incoming calls per unit time. A call center receiving an average of 3 calls per minute can use Poisson probability to calculate the likelihood of receiving 0, 1, 5, or 10 calls in the next minute, helping managers allocate staff optimally. Similarly, network engineers use it to model packet arrivals in data networks. In healthcare, hospital emergency departments use the Poisson model to predict patient arrivals per hour, enabling better resource allocation, bed management, and staffing decisions. Epidemiologists use it to study the incidence of rare diseases in a population.

In manufacturing and quality control, the Poisson distribution models the number of defects per unit of product — for example, the number of scratches per square meter of glass, or the number of printing errors per page. This allows manufacturers to set quality thresholds and make decisions about process improvements. In astronomy and physics, the distribution is applied to model the arrival of photons at a detector or the decay of radioactive particles. In traffic engineering, the number of vehicles passing a checkpoint per minute is modeled using Poisson. Insurance companies use it to estimate the number of claims per month in a region, helping set premium rates. Even in biology, Poisson processes describe the occurrence of DNA mutations per replication. In every case, the distribution provides a principled, mathematically tractable framework for reasoning about random, rare, and independent events.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import poisson

scenarios = [
    ("Call Center (calls/min)",   2),
    ("Hospital ER (patients/hr)", 5),
    ("Web Server (hits/sec)",    10),
    ("Defects per 1000 units",    3),
]
```

```
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
fig.suptitle("Q2 – Poisson Distribution: Real-World Scenarios", fontsize=14, fontweight='bold')
colors = ["steelblue", "seagreen", "tomato", "orchid"]

for ax, (title, lam), color in zip(axes.flatten(), scenarios, colors):
    x = np.arange(0, lam * 4)
    y = poisson.pmf(x, lam)

    ax.bar(x, y, color=color, alpha=0.8, edgecolor='black', linewidth=0.5)
    ax.axvline(lam, color="black", linestyle="--", linewidth=2, label=f"Mean = λ = {lam}")
    ax.set_title(title, fontsize=12, fontweight='bold')
    ax.set_xlabel("Count")
    ax.set_ylabel("Probability")
    ax.legend()
    ax.grid(True, alpha=0.3)

    p0   = poisson.pmf(0, lam)
    plam = poisson.pmf(lam, lam)
    print(f"{title}")
    print(f"   P(X=0)      = {p0:.4f}")
    print(f"   P(X=lambda) = {plam:.4f}")
    print(f"   Mean        = {lam}")
    print(f"   Variance    = {lam}")
    print()

plt.tight_layout()
plt.show()
```

```
Call Center (calls/min)
   P(X=0)      = 0.1353
   P(X=lambda) = 0.2707
   Mean        = 2
   Variance    = 2

Hospital ER (patients/hr)
   P(X=0)      = 0.0067
   P(X=lambda) = 0.1755
   Mean        = 5
   Variance    = 5

Web Server (hits/sec)
   P(X=0)      = 0.0000
   P(X=lambda) = 0.1251
   Mean        = 10
   Variance    = 10

Defects per 1000 units
   P(X=0)      = 0.0498
   P(X=lambda) = 0.2240
   Mean        = 3
   Variance    = 3
```



Q3 **What is the Central Limit Theorem and how does it relate to Normal distribution?**

ANS:- The Central Limit Theorem (CLT) is one of the most profound and far-reaching results in all of probability theory and statistics. It states that when a sufficiently large number of independent and identically distributed (i.i.d.) random variables are averaged together, the resulting distribution of sample means will approximate a Normal distribution, regardless of the shape of the original population distribution. Formally, if $X_1, X_2, \ldots, X_n$ are i.i.d. random variables with mean μ and finite variance σ^2, then the

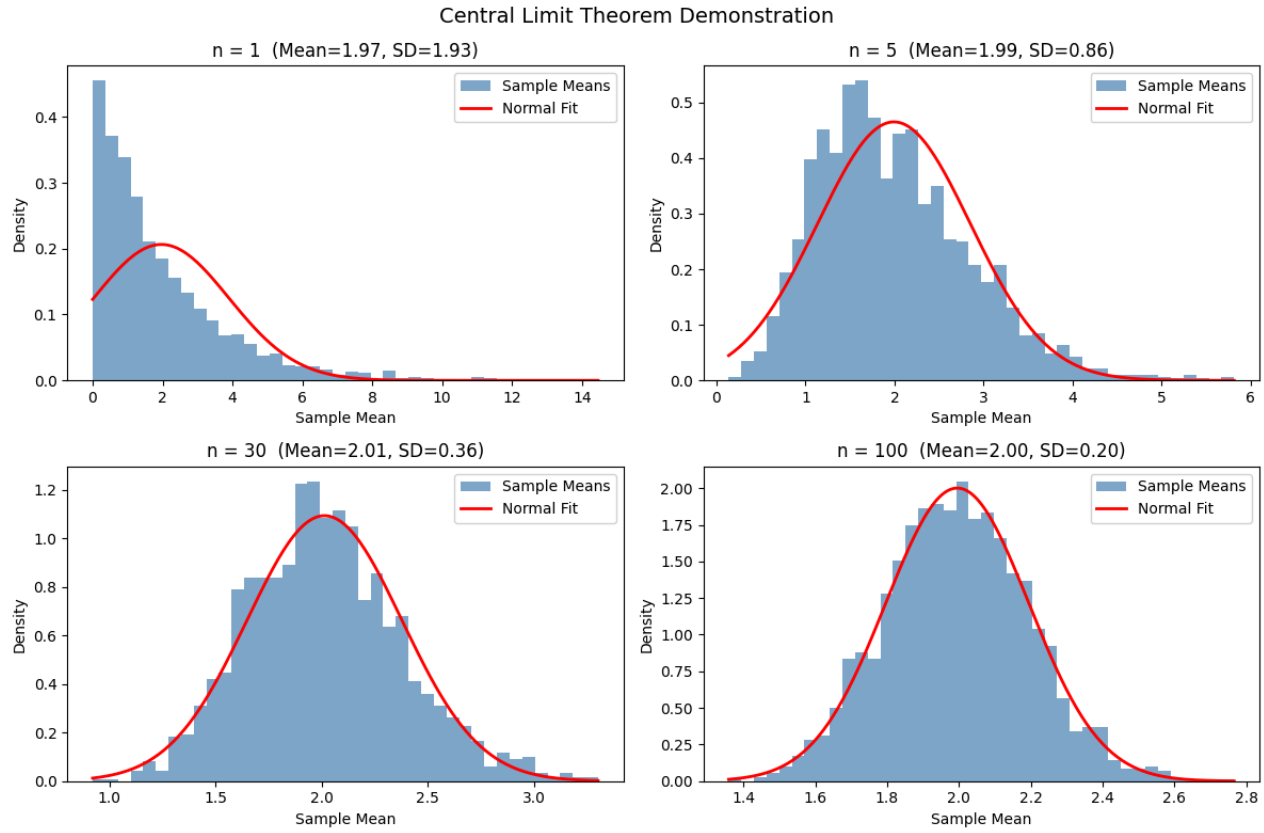
sample mean $\bar{X} = (X_1 + X_2 + \ldots + X_n)/n$ will follow an approximately Normal distribution $N(\mu, \sigma^2/n)$ as n approaches infinity. In practice, this approximation is considered good enough for $n \geq 30$, though this threshold varies with the skewness of the original distribution.

The CLT is the primary reason why the Normal distribution is so pervasive in statistics. Even when the underlying population is highly skewed — such as income distribution, exponential waiting times, or Poisson event counts — the distribution of sample means progressively becomes bell-shaped and symmetric as sample size grows. This means that statistical procedures built on the Normal distribution (such as t-tests, z-tests, confidence intervals, and regression analysis) can be validly applied to sample statistics even when the original data is non-normal, provided the sample is large enough. Without the CLT, inferential statistics would be far more limited in its applicability.

The relationship between the CLT and the Normal distribution is essentially a justification story: the Normal distribution is not merely an idealized mathematical construct — it is the natural limiting shape that emerges from the aggregation of many independent random influences. This is why so many natural measurements tend to be approximately Normally distributed. Human heights, for instance, are influenced by hundreds of genetic and environmental factors; their sum converges to Normal by the CLT. In the Python demonstration below, we sample from a highly skewed Exponential distribution and show how sample means progressively adopt a Normal shape as n increases from 1 to 100

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

np.random.seed(42)

population = np.random.exponential(scale=2, size=100_000)
print(f"Population shape: Exponential | Mean={population.mean():.2f}")

sample_sizes = [1, 5, 30, 100]

fig, axes = plt.subplots(2, 2, figsize=(12, 8))
fig.suptitle("Central Limit Theorem Demonstration", fontsize=14)

for ax, n in zip(axes.flatten(), sample_sizes):
    sample_means = [np.mean(np.random.choice(population, n))
                    for _ in range(2000)]

    ax.hist(sample_means, bins=40, density=True,
            color="steelblue", alpha=0.7, label="Sample Means")

    mu_fit = np.mean(sample_means)
    sd_fit = np.std(sample_means)
    x_fit = np.linspace(min(sample_means), max(sample_means), 200)
    ax.plot(x_fit, norm.pdf(x_fit, mu_fit, sd_fit),
            color="red", linewidth=2, label="Normal Fit")

    ax.set_title(f"n = {n}  (Mean={mu_fit:.2f}, SD={sd_fit:.2f})")
    ax.set_xlabel("Sample Mean")
    ax.set_ylabel("Density")
    ax.legend()

plt.tight_layout()
plt.show()
```

```
Population shape: Exponential | Mean=1.99
```



Q4 **Why is Normal distribution widely used in machine learning and statistics?**

ANS:- The Normal distribution occupies a uniquely central position in both classical statistics and modern machine learning, and its prevalence is no coincidence — it arises from a confluence of mathematical, theoretical, and empirical reasons that make it not only convenient but often the most principled choice when modeling data and designing algorithms.

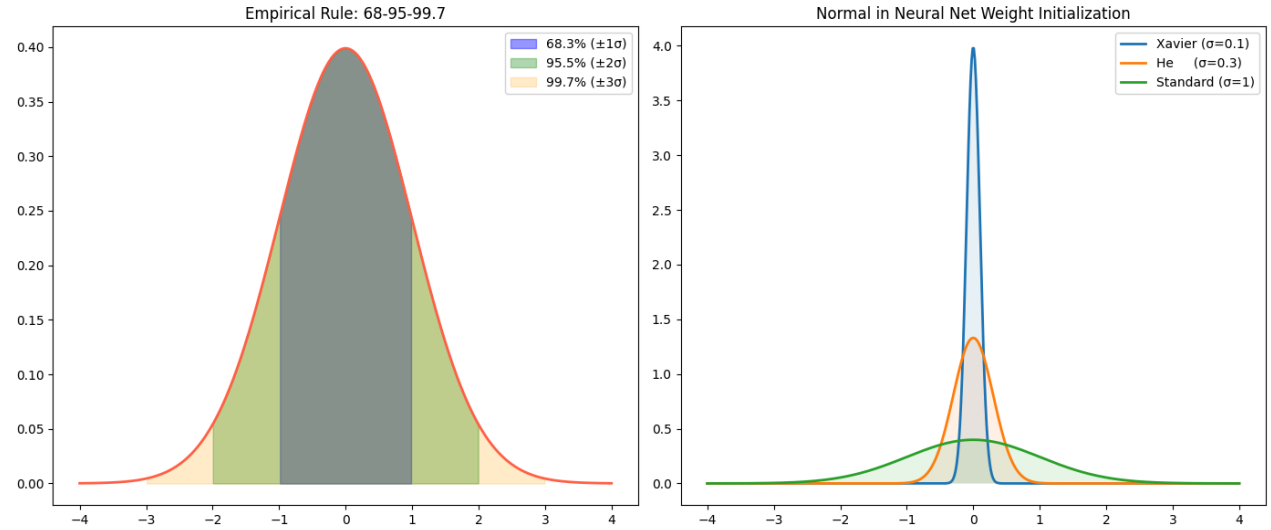
From a mathematical standpoint, the Normal distribution is extraordinarily tractable. It is entirely defined by just two parameters — the mean (μ) and the standard deviation (σ) — making it easy to interpret and work with. Crucially, any linear combination of Normal random variables is itself Normal, and the sum of two independent Normal distributions is also Normal. This "closure under addition" property makes algebraic manipulation simple and enables elegant analytical solutions in areas like Kalman filtering, Bayesian inference, and linear regression. The density function integrates to 1 over the entire real line and has well-known, easy-to-compute moments, derivatives, and integrals, all of which simplify the mathematics of likelihood estimation and optimization.

From a theoretical perspective, the Central Limit Theorem guarantees that the Normal distribution is the natural limiting distribution for the average of a large number of independent random variables. This means that regardless of the original distribution of the data — whether Poisson, Binomial, uniform, or something entirely non-standard — the sampling distribution of the mean will converge to Normal for large enough samples. This gives statisticians the confidence to apply Normal-based inference methods broadly. Furthermore, the maximum entropy principle states that among all distributions with a fixed mean and variance, the Normal distribution has the highest entropy, meaning it makes the fewest additional

assumptions about the data beyond what is already known. This makes it the most neutral, unbiased choice in the absence of other information.

In machine learning specifically, the Normal distribution plays roles at every level of model design. Linear regression assumes that residuals (errors) are Normally distributed, which underpins ordinary least squares estimation and the validity of confidence intervals and t-tests on coefficients. Neural network weights are commonly initialized using Normal distributions (such as Xavier or He initialization) to ensure stable gradient flow during training. Gaussian Naive Bayes classifiers model each feature as Normally distributed within each class. Gaussian Processes use the Normal distribution as the prior over functions for regression in a non-parametric setting. Gaussian Mixture Models (GMMs) decompose complex multivariate distributions into a mixture of Normals for clustering and density estimation. Variational Autoencoders (VAEs) encode the latent space as a Normal distribution, enabling smooth sampling and generation of new data. In all these contexts, the Normal distribution provides a mathematically principled, computationally efficient, and theoretically justified foundation.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

mu, sigma = 0, 1
x = np.linspace(-4, 4, 500)
y = norm.pdf(x, mu, sigma)

fig, axes = plt.subplots(1, 2, figsize=(14, 6))

ax = axes[0]
ax.plot(x, y, color="tomato", linewidth=2)
for sd, col, alpha, label in [
    (1, "blue",   0.4, "68.3% (±1σ)"),
    (2, "green",  0.3, "95.5% (±2σ)"),
    (3, "orange", 0.2, "99.7% (±3σ)"),
]:
    mask = (x >= -sd) & (x <= sd)
    ax.fill_between(x[mask], y[mask], alpha=alpha,
                    color=col, label=label)
ax.set_title("Empirical Rule: 68-95-99.7")
ax.legend()

ax = axes[1]
for label, s, col in [
    ("Xavier (σ=0.1)", 0.1, "steelblue"),
    ("He     (σ=0.3)", 0.3, "seagreen"),
    ("Standard (σ=1)", 1.0, "tomato"),
]:
    ax.plot(x, norm.pdf(x, 0, s), label=label, linewidth=2)
    ax.fill_between(x, norm.pdf(x, 0, s), alpha=0.1)
ax.set_title("Normal in Neural Net Weight Initialization")
ax.legend()

plt.tight_layout()
plt.show()

data = np.random.normal(0, 1, 100000)
for k in [1, 2, 3]:
    pct = np.mean(np.abs(data) <= k) * 100
    print(f"Within ±{k}σ: {pct:.2f}%")
```




```
Within ±1σ: 68.38%
Within ±2σ: 95.46%
Within ±3σ: 99.71%
```

Q5 **What is skewness in distributions and which distributions exhibit it?**

ANS:- Skewness is a statistical measure that quantifies the degree of asymmetry of a probability distribution relative to its mean. While a perfectly symmetric distribution, like the Normal distribution, has a skewness of zero, real-world data frequently exhibits imbalances where values are concentrated more on one side than the other, causing the distribution to have a longer tail extending either to the right or to the left. Formally, skewness is defined as the third standardized moment: Skewness = $E[(X - \mu)^3] / \sigma^3$. Raising the deviation to the third power (rather than the second, as in variance) preserves the sign of the deviation, allowing us to distinguish between left and right asymmetry. A large positive skewness means the right tail is longer, while a large negative skewness indicates a longer left tail.

Positive skewness (right skewness) occurs when the majority of data values are clustered toward the lower end of the scale, but a few extremely high values pull the mean to the right of the median. In such distributions, the relationship Mean > Median > Mode holds. Real-world examples are abundant: income and wealth distributions are strongly right-skewed because most people earn modest amounts while a small number of individuals earn enormously high incomes; the response time of web servers is right-skewed because most requests complete quickly but occasional slowdowns create a long right tail; and the duration of phone calls at a call center follows a similar pattern. Among named distributions, the Exponential, Poisson (with small λ), Chi-squared, Log-Normal, F-distribution, and Beta distribution (with $\alpha < \beta$) all exhibit positive skewness.

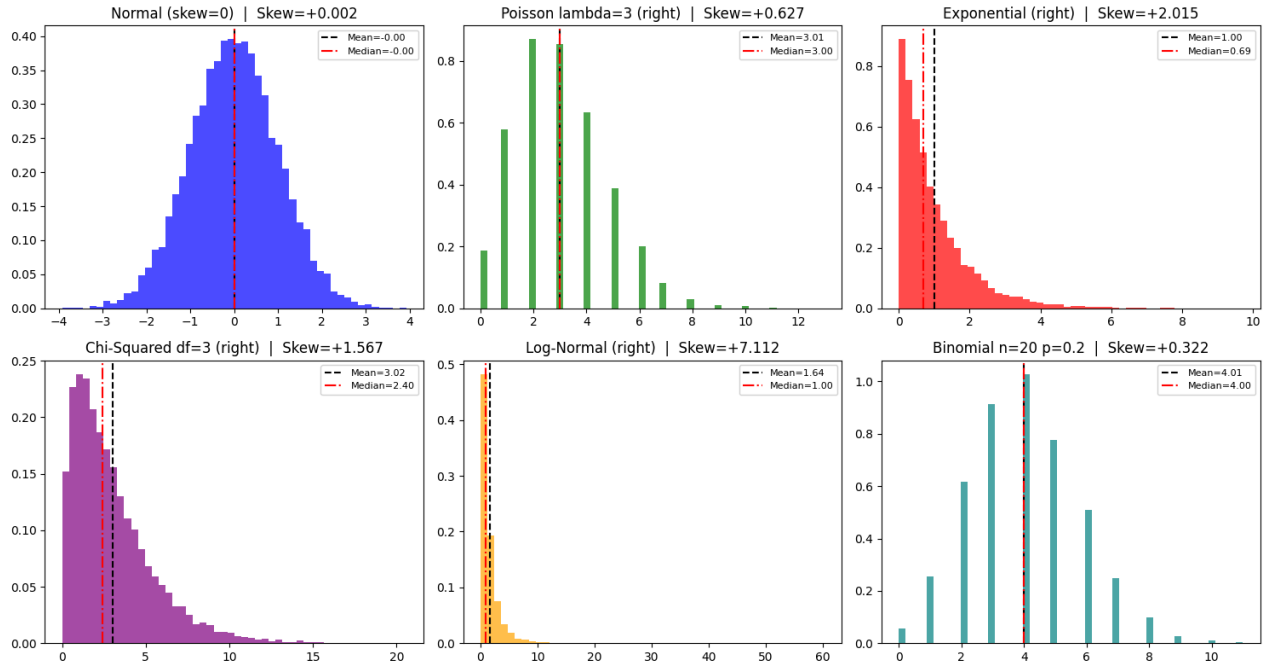
Negative skewness (left skewness) occurs when data is concentrated at the upper end of the scale, with a long left tail of unusually low values, so that Mean < Median < Mode. Examples include the age of death in a developed country, where most people live long lives but a few die young pulling the distribution left; or examination scores when the test is very easy and most students score near the maximum. In terms of named distributions, the Beta distribution with $\alpha > \beta$ and certain versions of the Weibull distribution exhibit negative skewness. The Normal distribution, with a skewness of exactly zero, serves as the reference point

for symmetry. The Binomial distribution has zero skewness when p = 0.5, but becomes positively skewed when p < 0.5 and negatively skewed when p > 0.5, with skewness formula (1-2p)/√(np(1-p)).

Understanding skewness has direct practical implications in data science and machine learning. Many algorithms — including linear regression, linear discriminant analysis, and Gaussian Naive Bayes — assume that features are approximately Normally distributed. Highly skewed features violate this assumption and can lead to poor model performance. Data scientists routinely apply transformations such as the logarithm, square root, or Box-Cox transformation to reduce skewness and bring data closer to Normal before training models. Skewness also affects the interpretation of the mean as a measure of central tendency: in a right-skewed distribution, the mean overestimates the "typical" value, and the median is a more robust measure. Detecting and correcting for skewness is therefore a standard step in exploratory data analysis and feature engineering pipelines.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

np.random.seed(42)

distributions = {
    "Normal (skew=0)":         np.random.normal(0, 1, 10000),
    "Poisson lambda=3 (right)":np.random.poisson(3, 10000),
    "Exponential (right)":     np.random.exponential(1, 10000),
    "Chi-Squared df=3 (right)":np.random.chisquare(3, 10000),
    "Log-Normal (right)":      np.random.lognormal(0, 1, 10000),
    "Binomial n=20 p=0.2":     np.random.binomial(20, 0.2, 10000),
}

print(f"{"Distribution":<30} {"Skewness":>10} {"Direction":>15}")
print("-" * 58)
for name, data in distributions.items():
    skew = stats.skew(data)
    direction = ("Right-Skewed" if skew > 0.1
                 else "Left-Skewed" if skew < -0.1
                 else "Symmetric")
    print(f"{name:<30} {skew:>+10.3f} {direction:>15}")

fig, axes = plt.subplots(2, 3, figsize=(15, 8))
colors = ["blue","green","red","purple","orange","teal"]

for ax, (name, data), color in zip(
        axes.flatten(), distributions.items(), colors):
    skew_val = stats.skew(data)
    ax.hist(data, bins=50, color=color, alpha=0.7, density=True)
    ax.axvline(np.mean(data), color="black",
               linestyle="--", label=f"Mean={np.mean(data):.2f}")
    ax.axvline(np.median(data), color="red",
               linestyle="-.", label=f"Median={np.median(data):.2f}")
    ax.set_title(f"{name}  |  Skew={skew_val:+.3f}")
    ax.legend(fontsize=8)

plt.tight_layout()
plt.show()
```

```
Distribution                      Skewness       Direction
-----------------------------------------------------------
Normal (skew=0)                     +0.002       Symmetric
Poisson lambda=3 (right)            +0.627    Right-Skewed
Exponential (right)                 +2.015    Right-Skewed
Chi-Squared df=3 (right)            +1.567    Right-Skewed
Log-Normal (right)                  +7.112    Right-Skewed
Binomial n=20 p=0.2                 +0.322    Right-Skewed
```



CONCLUSION:-

This assignment provided a thorough exploration of three foundational probability distributions — Binomial, Poisson, and Normal — and illuminated their distinct characteristics, mathematical properties, and real-world relevance. Through both theory and Python implementation, we discovered that each distribution serves a different purpose and is suited to a different type of data-generating process.

The Binomial distribution is the appropriate model whenever an experiment consists of a fixed number of independent binary trials with a constant success probability. Its discrete, finite nature makes it ideal for counting successes in well-defined repeated experiments such as quality control checks, clinical trial outcomes, and survey responses. The Poisson distribution, on the other hand, captures the behavior of rare, independent events occurring over a continuous interval of time or space. With its elegant single-parameter formulation and the unique property that mean equals variance, it is indispensable in telecommunications, healthcare logistics, manufacturing quality control, and any domain where random arrivals or occurrences must be modeled and predicted.

The Normal distribution stands as the cornerstone of both statistics and machine learning, not merely